

Complete CI/CD Setup Guide on AWS (Jenkins + SonarQube + Nexus + Tomcat + Maven)

This guide contains complete end-to-end DevOps setup steps collected from practical hands-on notes and AWS lab practice. It includes: • Jenkins Installation • Maven Setup • SonarQube Integration • Nexus Repository Setup • Apache Tomcat Deployment • CI/CD Jenkins Pipeline

1. AWS EC2 Instance Configuration

Recommended EC2 Type: t3.medium (2 vCPU, 4GB RAM)

OS: Ubuntu 22.04

Security Group Ports:

- 22 → SSH
- 8080 → Jenkins / Tomcat
- 9000 → SonarQube
- 8081 → Nexus
- 80 → HTTP

2. Jenkins Installation

Step 1: Connect to EC2 Instance

>nano Jenkins-installations.sh

```
#!/bin/bash
```

Install Java: `sudo apt update`

```
sudo apt install openjdk-21-jdk -y
```

Verify Java: `java -version`

Install Jenkins:

Copy Ubuntu/Debian commands from Jenkins official website.

```
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
```

```
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null"
```

```
sudo apt update
```

```
sudo apt install jenkins -y
```

Start Jenkins: `sudo systemctl enable jenkins`

```
sudo systemctl start jenkins
```

```
sudo systemctl status jenkins
```

Access Jenkins:

`http://<Public-IP>:8080`

3. Unlock Jenkins & Plugin Setup

Get Initial Password: `sudo cat /var/lib/jenkins/secrets/initialAdminPassword`

Paste the password in browser and install suggested plugins.

```
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 28
ubuntu@ip-172-31-144-155:~$ systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-05-17 14:54:05 UTC; 4min 33s ago
 Invocation: 17de142ef9fb42f09e3c8b4c427be25e
   Main PID: 4538 (java)
    Tasks: 43 (limit: 3712)
   Memory: 702.7M (peak: 715.8M)
      CPU: 16.681s
   CGroup: /system.slice/jenkins.service
           └─4538 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webro

May 17 14:54:00 ip-172-31-144-155 jenkins[4538]: [LF]> This may also be found at: /var/lib/jenkins
May 17 14:54:00 ip-172-31-144-155 jenkins[4538]: [LF]>
May 17 14:54:00 ip-172-31-144-155 jenkins[4538]: [LF]> *****
May 17 14:54:00 ip-172-31-144-155 jenkins[4538]: [LF]> *****
May 17 14:54:00 ip-172-31-144-155 jenkins[4538]: [LF]> *****
May 17 14:54:05 ip-172-31-144-155 jenkins[4538]: 2026-05-17 14:54:05.728+0000 [id=40]          INFO
May 17 14:54:05 ip-172-31-144-155 jenkins[4538]: 2026-05-17 14:54:05.754+0000 [id=30]          INFO
May 17 14:54:05 ip-172-31-144-155 systemd[1]: Started jenkins.service - Jenkins Continuous Integra
May 17 14:54:07 ip-172-31-144-155 jenkins[4538]: 2026-05-17 14:54:07.462+0000 [id=56]          INFO
May 17 14:54:07 ip-172-31-144-155 jenkins[4538]: 2026-05-17 14:54:07.462+0000 [id=56]          INFO
ubuntu@ip-172-31-144-155:~$ sudo cat /var/lib/jenkins/secrets/initialAdminPassword
3f087b90dea34340bac44e5b37ce9b01
ubuntu@ip-172-31-144-155:~$ ^C
ubuntu@ip-172-31-144-155:~$ mvn -v
```

Important Plugins:

- Maven Integration
- SonarQube Scanner
- Nexus Artifact Uploader
- Pipeline
- Git
- Deploy to Container

4. Maven Installation & Configuration

Install Maven: `sudo apt install maven -y`

Verify: `mvn -version`

Configure in Jenkins:

Manage Jenkins → Tools → Maven Installations

Add Maven → Name: Maven-Home

Provide Maven Path from EC2 instance.

Maven installations

[+ Add Maven](#)

Maven

Name

maven-apache

MAVEN_HOME

/usr/share/maven

☐ Install automatically [?](#)

[+ Add Maven](#)

[Save](#) [Apply](#)

5. Jenkins Pipeline Example

```
pipeline {
  agent any

  stages {

    stage('Git Checkout') {
      steps {
        git branch: 'master', url: 'YOUR_GITHUB_REPO'
      }
    }

    stage('Compile') {
      steps {
        sh 'mvn compile'
      }
    }

    stage('Test') {
      steps {
        sh 'mvn test'
      }
    }

  }
}
```

6. SonarQube Setup

Create Separate EC2 for SonarQube

Install Java: `sudo apt update`
`sudo apt install openjdk-21-jdk -y`

Download SonarQube:

Copy latest Linux zip link from SonarQube website.

```
wget -O sonar.zip YOUR_LINK
sudo apt install unzip -y
unzip sonar.zip
```

Start SonarQube: `cd sonarqube/bin/linux-x86-64`

```
./sonar.sh start
./sonar.sh status
```

```
inflating: sonarqube-26.5.0.122743/lib/sonar-shutdowner-26.5.0.122743.jar
creating: sonarqube-26.5.0.122743/elasticsearch/plugins/
ubuntu@ip-172-31-197-35:~$ ls
sonar.zip  sonarqube-26.5.0.122743  sonarqube-installations.sh
ubuntu@ip-172-31-197-35:~$ cd sonarqube-26.5.0.122743/
ubuntu@ip-172-31-197-35:~/sonarqube-26.5.0.122743$ ls
COPYING  bin  conf  data  elasticsearch  extensions  jres  lib  licenses  logs  security  sonarsonar
ubuntu@ip-172-31-197-35:~/sonarqube-26.5.0.122743$ cd bin
ubuntu@ip-172-31-197-35:~/sonarqube-26.5.0.122743/bin$ ls
elasticsearch  linux-x86-64  macosx-universal-64  windows-x86-64  winsw-license
ubuntu@ip-172-31-197-35:~/sonarqube-26.5.0.122743/bin$ cd linux-x86-64
ubuntu@ip-172-31-197-35:~/sonarqube-26.5.0.122743/bin/linux-x86-64$ ls
sonar.sh
ubuntu@ip-172-31-197-35:~/sonarqube-26.5.0.122743/bin/linux-x86-64$ ./sonar.sh
/usr/bin/java
Usage: ./sonar.sh { console | start | stop | force-stop | restart | status | dump }
ubuntu@ip-172-31-197-35:~/sonarqube-26.5.0.122743/bin/linux-x86-64$ ./sonar.sh status
/usr/bin/java
SonarQube is not running.
ubuntu@ip-172-31-197-35:~/sonarqube-26.5.0.122743/bin/linux-x86-64$
ubuntu@ip-172-31-197-35:~/sonarqube-26.5.0.122743/bin/linux-x86-64$ ./sonar.sh start
/usr/bin/java
Starting SonarQube...
Started SonarQube.
ubuntu@ip-172-31-197-35:~/sonarqube-26.5.0.122743/bin/linux-x86-64$ ./sonar.sh status
/usr/bin/java
SonarQube is running (4379).
ubuntu@ip-172-31-197-35:~/sonarqube-26.5.0.122743/bin/linux-x86-64$
```

Access:

`http://<Public-IP>:9000`

Default Credentials:

Username: admin
Password: admin

7. Integrating SonarQube with Jenkins

Generate Token:

SonarQube → My Account → Security → Generate Token

Add Credentials in Jenkins:

Manage Jenkins → Credentials → Add Credentials

Type: Secret Text

Paste Sonar Token

Configure SonarQube Server:

Manage Jenkins → System → SonarQube Servers

Add Server URL and Token.

Pipeline Stage:

```
stage('Sonar Analysis') {
  steps {
    withSonarQubeEnv('sonar-server') {
      sh 'mvn sonar:sonar'
    }
  }
}
```

Workspace

Workspace / target /

→

classes

generated-sources/annotations

generated-test-sources/test-annotations

jpetstore

maven-archiver

osgi

site

sonar

surefire-reports

test-classes/org/mybatis/jpetstore

formatter-maven-cache.properties

import-maven-cache.properties

jacoco.exec

jpetstore.war

May 17, 2026, 4:06:39 PM

May 17, 2026, 4:06:38 PM

May 17, 2026, 4:07:15 PM

May 17, 2026, 4:07:16 PM

7.94 KiB

4.00 KiB

196.40 KiB

11.37 MiB

👁

👁

👁

👁

📁 (all files in zip)

```
projectKey=org.mybatis:jpetstore
serverUrl=http://13.202.100.230:9000
serverVersion=26.5.0.122743
dashboardUrl=http://13.202.100.230:9000/dashboard?id=org.mybatis%3Ajpetstore
ceTaskId=de1482f0-840e-4473-a8d4-6c584ddb28d9
ceTaskUrl=http://13.202.100.230:9000/api/ce/task?id=de1482f0-840e-4473-a8d4-6c584ddb28d9
```

⚠ Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. [Learn more](#)

📘 The way in which security, reliability, and maintainability counts and ratings are calculated has changed. [Learn more in SonarQube documentation](#)



ProjectsIssuesRulesQuality profilesQuality gatesAdministrationMore

My FavoritesAll

Filters

Quality Gate

✓ Passed1

✗ Failed0

Security

A ≥ 0 info issues1

B ≥ 1 low issue0

C ≥ 1 medium issue0

D ≥ 1 high issue0

E ≥ 1 blocker issue0

Reliability

A ≥ 0 info issues0

🔍 Search projects

PerspectiveOverall StatusSort byName

Create Project

1 project(s)

☆ JPetStore Demo 6Public

✓ Passed

Last analysis: 2 minutes ago • 3.4k Lines of Code • Java, JSP, ...

A0

C59

A61

A—

68.9%

4.6%

Security

Reliability

Maintainability

Hotspots Reviewed

Coverage

Duplications

1 of 1 shown

SonarQube™ technology is powered by SonarSource Sàrl

Community Build • v26.5.0.122743 • MQR MODE

LGPL v3CommunityDocumentationPluginsWeb

8. Nexus Repository Setup

Create Separate EC2 for Nexus

Install Java: `sudo apt update`
`sudo apt install openjdk-21-jdk -y`

Download Nexus: `wget -O nexus.tar.gz YOUR_NEXUS_LINK`
`tar -xvzf nexus.tar.gz`

Start Nexus: `cd nexus-*/bin`
`./nexus start`

```
nexus-3.92.2-01/jdk/temurin_21.0.9_10_linux_x86_64/jdk-21.0.9+10/bin/jdeprscan
nexus-3.92.2-01/jdk/temurin_21.0.9_10_linux_x86_64/jdk-21.0.9+10/bin/java
nexus-3.92.2-01/jdk/temurin_21.0.9_10_linux_x86_64/jdk-21.0.9+10/bin/rmiregistry
nexus-3.92.2-01/jdk/temurin_21.0.9_10_linux_x86_64/jdk-21.0.9+10/bin/serialver
nexus-3.92.2-01/jdk/temurin_21.0.9_10_linux_x86_64/jdk-21.0.9+10/bin/jstat
nexus-3.92.2-01/jdk/temurin_21.0.9_10_linux_x86_64/jdk-21.0.9+10/bin/jconsole
nexus-3.92.2-01/jdk/temurin_21.0.9_10_linux_x86_64/jdk-21.0.9+10/bin/jpackage
nexus-3.92.2-01/jdk/temurin_21.0.9_10_linux_x86_64/jdk-21.0.9+10/bin/jshell
nexus-3.92.2-01/jdk/temurin_21.0.9_10_linux_x86_64/jdk-21.0.9+10/bin/jimage
nexus-3.92.2-01/jdk/temurin_21.0.9_10_linux_x86_64/jdk-21.0.9+10/bin/jdeps
nexus-3.92.2-01/jdk/temurin_21.0.9_10_linux_x86_64/jdk-21.0.9+10/bin/keytool
nexus-3.92.2-01/jdk/temurin_21.0.9_10_linux_x86_64/jdk-21.0.9+10/bin/javac
nexus-3.92.2-01/jdk/temurin_21.0.9_10_linux_x86_64/jdk-21.0.9+10/bin/javadoc
ubuntu@ip-172-31-233-193:~$ ls
nexus-3.92.2-01  nexus-installations.sh  nexus.tar.gz  sonatype-work
ubuntu@ip-172-31-233-193:~$ cd nexus-3.92.2-01
ubuntu@ip-172-31-233-193:~/nexus-3.92.2-01$ ls
NOTICE.txt  OSS-LICENSE.txt  bin  deploy  etc  jdk
ubuntu@ip-172-31-233-193:~/nexus-3.92.2-01$ cd bin
ubuntu@ip-172-31-233-193:~/nexus-3.92.2-01/bin$ ls
nexus  nexus.vmoptions  sonatype-nexus-repository-3.92.2-01.jar
ubuntu@ip-172-31-233-193:~/nexus-3.92.2-01/bin$ cd nexus
-bash: cd: nexus: Not a directory
ubuntu@ip-172-31-233-193:~/nexus-3.92.2-01/bin$ ./nexus start
Starting nexus
ubuntu@ip-172-31-233-193:~/nexus-3.92.2-01/bin$ sudo cat /home/ubuntu/sonatype-work/nexus3/admin.password
6af5f461-0a20-4cb8-8739-bb3889eblad6ubuntu@ip-172-31-233-193:~/nexus-3.92.2-01/bin$
```

Access Nexus:
`http://<Public-IP>:8081`

9. Configure Nexus Repository

Initial Password: `sudo cat /opt/sonatype-work/nexus3/admin.password`

Create Repository:
Settings → Repositories → Create Repository
Choose Maven (hosted)

Install Jenkins Plugin:
Nexus Artifact Uploader Plugin

Add Nexus Credentials in Jenkins.

10. Upload Artifact to Nexus

```
stage('Upload Artifact') {
    steps {
        nexusArtifactUploader(
            nexusVersion: 'nexus3',
            protocol: 'http',
            nexusUrl: 'PUBLIC-IP:8081',
            groupId: 'com.example',
            version: '1.0',
            repository: 'maven-hosted',
            credentialsId: 'nexus-cred',
            artifacts: [[
                artifactId: 'demo-app',
                classifier: '',
                file: 'target/app.jar',
                type: 'jar'
            ]]
        )
    }
}
```

11. Apache Tomcat Setup

Create Tomcat EC2 Instance

Install Java: `sudo apt update`
`sudo apt install openjdk-21-jdk -y`

Download Tomcat: `wget https://downloads.apache.org/tomcat/tomcat-10/v10.1.54/bin/apache-tomcat-10.1.54.tar.gz`

Extract: `tar -xvzf apache-tomcat-10.1.54.tar.gz`

Navigate: `cd apache-tomcat-10.1.54/bin`

Start Tomcat: `chmod +x *.sh`
`./startup.sh`

Check Status: `ps -ef | grep tomcat`

Access:
`http://<Public-IP>:8080`

12. Deploy WAR File into Tomcat

Copy WAR File: `cp target/jpetstore.war ~/tomcat/webapps/`

Restart Tomcat: `cd ~/tomcat/bin`

`./shutdown.sh`

`./startup.sh`

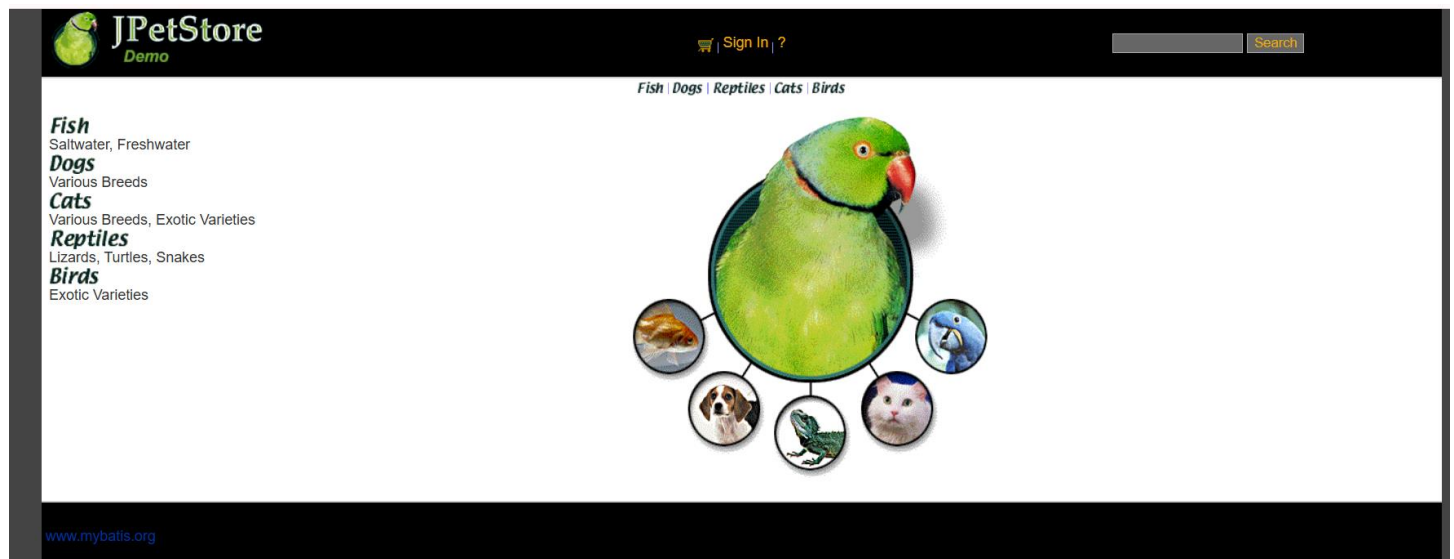
Application URL:

`http://<Public-IP>:8080/jpetstore`

Welcome to JPetStore 6

[Enter the Store](#)

Copyright www.mybatis.org



13. Full CI/CD Flow

GitHub Repository → Jenkins Pipeline → Maven Build → SonarQube Analysis → Nexus Artifact Upload → Tomcat Deployment

This creates a complete automated DevOps CI/CD pipeline using AWS infrastructure

14. Best Practices

- Use separate EC2 instances for Jenkins, SonarQube, Nexus, and Tomcat.
- Always open required ports in Security Groups.
- Use IAM Roles instead of access keys whenever possible.
- Keep Jenkins plugins updated.
- Store credentials securely in Jenkins Credentials Manager.
- Use Git branches for feature development.
- Enable automated backups for Nexus repositories

